Automatic Correction of Programming Exercises with Artemis & GitLab CI

TUMKolleg

Supervisor: Prof. Dr. Stephan Krusche
Advisors: Jan Philip Bernius, M.Sc., Matthias Linhuber, B.Sc.
Author: Benjamin Sebastian Schmitz

April 6, 2022

Abstract

Motivation/Objective: Since some Version Control (VC) systems include a Continuous Integration (CI) application, it is possible to reduce the effort required to maintain the servers and migrate the CI service from external applications as Jenkins to the inbuilt CI service.

Problem Statement: The interactive learning platform Artemis currently only supports setups with an external CI server. The CI server should be migrated to the VC server. Furthermore, it should be possible to adjust the build plan after the task has been assigned.

Proposed Solution: GitLab CI is the CI system provided by the GitLab VC server. This system supports the provision of the build plan via the Artemis API, which makes it possible to update the build plan retrospectively, as the build plan is fetched from the API on each run.

(Expected) Results: We expect to use the bridge pattern to dynamically bind Artemis to the concrete implementation with GitLab CI. The reduction in server load will mean that more universities, especially smaller ones with less capacity, will choose Artemis.

1 Problem Statement

The interactive learning platform Artemis is an approach to use interactive learning during large computer science classes [Krusche17, Krusche18]. Main goal is to give the students immediately individual and automatic generated feedback [Krusche18]. To achieve this goal, Artemis uses a Version Control (VC) and Continuous Integration (CI) server, to execute test cases predefined by the instructor on each submission of each student, display the build results and grade the submission [Krusche18].

Artemis currently supports Bamboo[1] and Jenkins[2] as a CI server for executing the tests. This leads to additional server overhead, since the CI and the Version Control (VC) are running on different servers. Particularly smaller universities might then not use Artemis.

Additionally, the base build plan is copied once after the student started the exercise [Krusche18]. Later changes to the base build plan are not patched automatically to all student build plans, but rather the build plans must be updated manually in the CI Server. Especially in large software engineering courses this is nearly impossible.

In exams, it would be useful to change the student build plans after the submission date to firstly hide the test cases during the exam and only compile the code and secondly use a debug compiler not failing due to a simple syntax error. The current setup and implementation makes it very cumbersome to update the build plan afterwards.

2 Proposed Solution

The solution for the problem described in section 1 is a CI system not storing each build plan individually, but each time the code in the VC Server gets updated, the build plan is fetched. It could be transposed with a CI server which is dynamically updating the student build plan. Furthermore, the CI service should be part of the VC server to reduce additional effort by maintaining two servers.

The Version Control server GitLab[3] has an integrated CI service called GitLab CI[4]. The advantage of this application is the possibility to configure the build plan as an online available file[5]. The build plan can be made available via the API of Artemis and GitLab CI is downloading the build plan for each execution of the test cases. As a result, instructors are able to update the build plan after the release of an exercise, GitLab CI fetches the updated build plan when needed. This process is illustrated in section 4.

3 Related Work

According to Grady Booch's book *Object-oriented analysis and design with applications* integration should take generally take place regularly, to avoid problems later on [Booch95]. Martin Fowler explains in his post *Continuous Integration*, Continuous Integration as a practice, in which each integration, which should take place at least once per day per developer, is build and tested with predefined test cases[6].

In their paper *Interactive Learning – Increasing Student Participation through Shorter Exercise Cycles*, Krusche et al. have taken up this concept to improve the learning experience in large computer science courses at university [Krusche17]. They introduced an newly developed learning method which shortens the time between theory, which is usually taught in the non-interactive lecture, and practice, which typically follows about a week later, so that less knowledge is lost [Krusche17]. The concept, called Interactive learning, makes it possible for students to transfer the new acquired skills to real, application-based use cases still during lecture [Krusche17]. In order to implement this idea, the lecture is split up

[1] `https://www.atlassian.com/software/bamboo`, accessed on March 09, 2022
[2] `https://www.jenkins.io/`, accessed on March 09, 2022
[3] `https://about.gitlab.com/`, accessed on March 09, 2022
[4] `https://about.gitlab.com/stages-devops-lifecycle/continuous-integration/`, accessed on March 09, 2022
[5] `https://docs.gitlab.com/`, accessed on March 09, 2022
[6] `https://martinfowler.com/articles/continuousIntegration.html`, accessed on March 19, 2022

into multiple iterations, each consisting of different phases [Krusche17]. First there is a theory phase, before the instructor shows an example to the students. After that the students can try out the concept themselves, before they get a sample solution and instant, but still individual feedback generated by the CI server. Afterwards it is possible to reflect about the newly learned approach. With this fundamental idea, later the assessment management system Artemis was built [Krusche18].

Hilton et al. verify in their paper *Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects* that using CI in open source software is useful to prevent bugs and build failures [Hilton16]. Not only projects using CI release twice as often as projects without CI, but the build is also less likely to fail in the master branch [Hilton16].

Over the past few years Continuous Integration has become far more accessible for even hobby programmers, since CI services are now often included with VC servers [7]. This is why it is now possible to migrate external CI servers as Jenkins to GitLab CI and so reduce additional work for server administration.

Since the benefits of using CI are quite high and nearly half of the developers who do not use CI mention a lack of familiarity as the reason for not using a CI service, it is useful to teach the concepts of CI at university [Hilton16].

4 System Requirements and Top Level Design

The visionary scenario presents the supposed workflow of an exercise.

1) Name: Exercise work flow

2) Participating actors: Bob: Instructor, Mary: Student

3) Flow of events:

1. Bob prepares the exercise via the web interface of Artemis and pushes the exercise code template, the test cases as well as the sample solution to the VC server [Krusche18].
2. Artemis configures the build plan accordingly to the configuration made by Bob and makes it available via the API.
3. Mary clicks the start exercise button after the release date [Krusche18].
4. The student base repository is copied [Krusche18].
5. GitLab CI gets configured so that it takes the build plan from the Artemis API.
6. Mary clones the repository to her local device [Krusche18].
7. Mary solves the exercise [Krusche18].
8. Mary commits and pushes her adapted code [Krusche18].
9. The GitLab CI build gets triggered [Krusche18].
10. GitLab CI fetches the build plan from the API and executes the test cases according to the plan.
11. GitLab CI sends the build results to Artemis [Krusche18].
12. Artemis displays the build results [Krusche18].
13. Mary reviews her result in Artemis.
14. After the submission Bob can review the course results and adapt the lecture appropriate to the skills of the students [Krusche18].

Using the problem statement section 1 and the visionary scenario, it is possible to determine the relevant actors and systems for solving the problem.

[7] `https://about.gitlab.com/releases/2015/09/22/gitlab-8-0-released/`, accessed on March 16, 2022

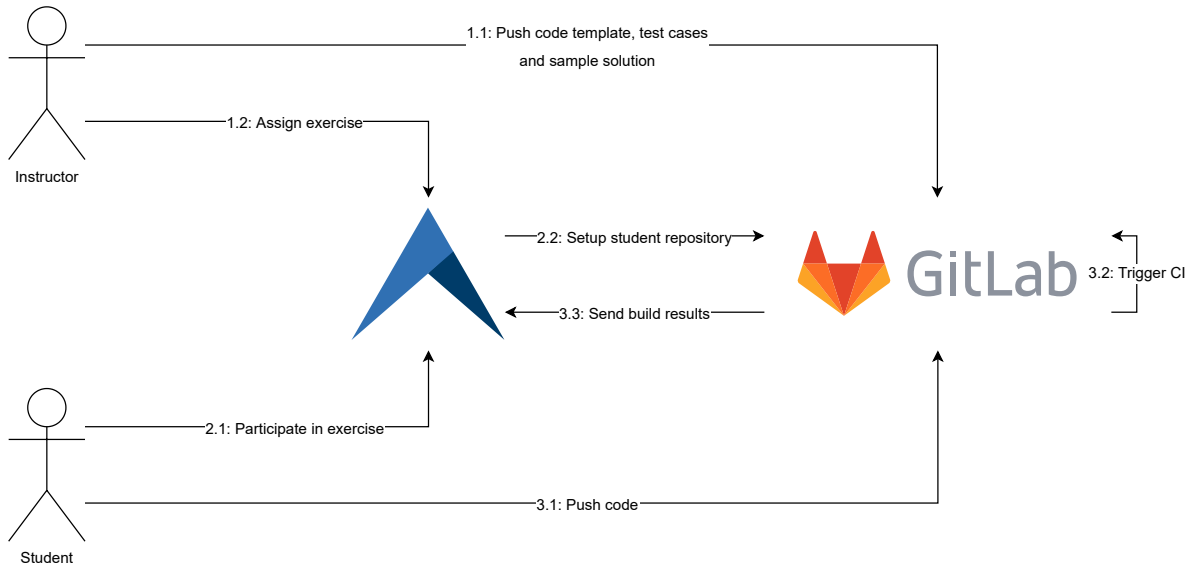


Figure 1: Depiction of the top-level workflow of the interactions in relation to Artemis and GitLab CI

5 Expected Findings

This research thesis deals with the question on how to connect Artemis with GitLab CI. We expect to use the bridge pattern, described by Gamma et al. in their book *Design Patterns: Elements of Reusable Object-Oriented Software*, for dynamically choosing the implementation with GitLab CI during startup of the system [Gamma95]. For making the build plan accessible, the API of Artemis might be extended for this service.

On long term, we expect that more universities will use Artemis, since the setup hurdle will become smaller. As described in section 3, this might lead to better developers and better code, due to the fact, that more students will be taught the concepts of CI.

In addition, universities will save time and money as they will need at least one less server.

The proposed solution opens up new opportunities for further research. with the envisaged implementation it will be possible to tackle the problem regarding the test execution during exams described in section 1. In addition, either Ansible or a Kubernetes cluster can be used for automatic deployment of the GitLab CI Runners.

6 Time Schedule

The following assumptions were made for the preparation of the time schedule:

- Timeframe: March to December 6, 2022
- Every week, Wednesday is set aside for working on the thesis.
- During July there will be an internship as part of the TUMKolleg programm. During this time, I will not have time to work on my thesis. Instead, I plan to work on my thesis for about 2 weeks during the summer holidays in August.

It is planned to adhere to the following time schedule:

- March

  - 5th week: Proposal
- April
  - 2nd week: Docker GitLab CI Runner configuration
  - 4th week: prototype implementation: Connectivity to GitLab CI
- May
  - 3nd week: prototype implementation: Artemis API for configuration
- June
  - 4th week: prototype implementation: Sending build results to Artemis
  - 4th week: Models (parallel to prototype implementation)
  - 4th week: Requirements (parallel to prototype implementation)
- July: *Internship*
- August: *Summer holidays*
- September
  - 2nd week: Documentation for installation and configuration of Artemis with GitLab CI
  - 2nd week: Revise models with concrete implementation
- October: *Writing ...*
- November
  - 2nd week: Thesis ready for review
  - 3rd week: Review finished
  - 4th week: Implement improvements
- December
  - 1st week: Thesis

References

[Booch95] Booch, G. *Object-oriented analysis and design with applications (2. ed.)*. Benjamin/Cummings series in object-oriented software engineering. Addison-Wesley. 1995. ISBN 978-0-8053-5340-2.

[Gamma95] Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Longman Publishing Co., Inc., USA. 1995. ISBN 0201633612.

[Hilton16] Hilton, M.; Tunnell, T.; Huang, K.; Marinov, D.; Dig, D. Usage, costs, and benefits of continuous integration in open-source projects. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2016, pp. 426–437.

[Krusche17] Krusche, S.; Seitz, A.; Börstler, J.; Brügge, B. Interactive learning: Increasing student participation through shorter exercise cycles. In *Proceedings of the Nineteenth Australasian Computing Education Conference, ACE 2017, Geelong, VIC, Australia, January 31 - February 3, 2017*. ACM. 2017, pp. 17–26.

[Krusche18] Krusche, S.; Seitz, A. ArTEMiS: An Automatic Assessment Management System for Interactive Learning. ACM. 2018, 49th Technical Symposium on Computer Science Education, pp. 284–289.